

LibriSpeech 100 小时语音识别实验：BiLSTM-CTC 与 Transformer-CTC 对比

1. 实验目的

本实验在 LibriSpeech 英文语音数据集上实现端到端自动语音识别（Automatic Speech Recognition, ASR），比较 BiLSTM 与 Transformer 两种声学编码网络在相同实验条件下的识别效果。

两种模型均采用字符级 CTC（Connectionist Temporal Classification）训练，不需要预先获得音频帧与字符之间的对齐关系。实验的主要评价指标为：

- **WER (Word Error Rate)**：错词率，越低越好；
- **CER (Character Error Rate)**：字符错误率，越低越好。

为了尽可能公平地比较两种网络，二者使用完全相同的数据划分、声学特征、字符词表、CTC 目标、贪心解码方法以及误差率计算方法，模型参数量也控制在约 410 万。

2. 数据集与实验划分

实验采用 LibriSpeech 的 clean 子集，服务器上的数据位于：

```
1 | /scratch/zhanz720/library_speech/dataset/LibriSpeech
```

各数据划分如下。

数据划分	用途	语音条数	总时长	说话人数
train-clean-100	模型训练	28,539	100.591 小时	251
dev-clean	选择最佳权重、观察泛化性能	2,703	5.388 小时	40
test-clean	最终一次性测试	2,620	5.403 小时	40

训练使用完整的 train-clean-100，没有再进行小时数抽样。训练过程中根据 dev-clean 上的 WER 保存最佳权重。所有训练与模型选择结束后，才在 test-clean 上进行统一的最终测试。因此，test-clean 没有参与训练、学习率调整、提前停止或最佳模型选择。

3. 输入特征、输出形式与 CTC

3.1 输入声学特征

原始 FLAC 音频首先读取为单声道波形，并统一到 16 kHz 采样率，然后提取 80 维 log-Mel 频谱特征：

配置项	数值
采样率	16,000 Hz
Mel 滤波器数量	80
FFT 点数	400
窗长	400 个采样点 (25 ms)
帧移	160 个采样点 (10 ms)

每条语音分别进行均值和标准差归一化。模型输入可以表示为长度可变的特征序列：

```
1 | [时间帧数 T, Mel 特征维度 80]
```

3.2 输出字符集合

模型使用 29 个输出类别：

```
1 | CTC blank + 撇号 + 空格 + A 到 Z
```

文本统一转换为大写，只保留英文字母、撇号和空格。网络在每个降采样后的时间步输出 29 维 logits，经过 `log_softmax` 后传给 CTC Loss。

从分类含义上看，每个时间步都是一个 29 类 one-hot 目标问题；但实际实现中不显式创建 one-hot 向量，而是按照 PyTorch `CTCLoss` 的要求，将目标文本保存为字符类别的整数 ID。这样与 one-hot 分类语义等价，同时更节省内存。

3.3 CTC 解码

本实验采用 greedy CTC 解码：

1. 在每个时间步选择概率最大的字符；
2. 合并连续重复字符；
3. 删除 CTC blank；
4. 得到最终识别文本。

实验没有使用 beam search、外部语言模型或预训练声学模型，因此结果反映的是两个基础网络本身的建模能力。

4. 模型结构

两种模型共享相同的卷积前端和字符级 CTC 输出层，主要区别是中间的时序编码器。

整体数据流如下：

```
1 | 16 kHz 音频
2 |   → 80 维 log-Mel 特征
3 |   → 两层一维卷积降采样
4 |   → BiLSTM 或 Transformer 编码器
5 |   → 29 类字符输出
6 |   → CTC Loss / greedy CTC 解码
```

4.1 共享卷积前端

共享前端包含两层一维卷积：

```
1 Conv1d(80 → 128, kernel_size=5, stride=2, padding=2) + ReLU
2 Conv1d(128 → 128, kernel_size=5, stride=2, padding=2) + ReLU
```

两层卷积共将时间长度压缩约 4 倍。这样既可以减少后续网络的计算量，也能够进入时序编码器前融合相邻声学帧的信息。

4.2 BiLSTM-CTC

BiLSTM 模型结构为：

- 共享的两层卷积前端；
- 3 层双向 LSTM；
- 每个方向的隐藏维度为 256；
- LSTM 层间 dropout 为 0.2；
- 双向输出维度为 512；
- 线性分类层：512 → 29；
- 总参数量：4,092,701。

双向 LSTM 同时利用当前帧之前和之后的声学上下文，适合离线语音识别任务。LSTM 的循环结构还带有较强的局部顺序归纳偏置，在数据规模有限时通常比较容易训练。

4.3 Transformer-CTC

Transformer 模型结构为：

- 共享的两层卷积前端；
- 输入投影层：128 → 256；
- 正弦位置编码；
- 输入 dropout 为 0.1；
- 5 层 Pre-LN Transformer Encoder；
- 模型维度 `d_model = 256`；
- 4 个注意力头；
- 前馈网络维度为 1,024；
- 激活函数为 GELU；
- 最终 LayerNorm；
- 线性分类层：256 → 29；
- 总参数量：4,123,165。

Transformer 通过自注意力直接建模不同时间位置之间的关系。由于注意力本身不包含序列顺序信息，因此加入正弦位置编码。Padding mask 用于阻止模型关注批次中补齐的无效时间帧。

两种模型的参数量只相差 30,464，差异约为 0.74%，因而可以近似视为同等模型规模下的结构对比。

5. 训练方案

训练在 Alliance Canada 计算集群的一张 NVIDIA H100 80GB HBM3 GPU 上完成，Python 环境中的主要软件版本为：

```

1 Python 3.11.5
2 PyTorch 2.11.0+computeCanada
3 torchaudio 2.11.0+computeCanada
4 soundfile 0.13.1+computeCanada

```

5.1 共同训练设置

配置项	设置
训练集	完整 train-clean-100
验证集	完整 dev-clean
最大 epoch	30
batch size	8
随机种子	7
损失函数	CTC Loss
优化器	AdamW
初始/峰值学习率	0.001
最佳模型标准	最低 dev WER; WER 相同时选择更低 dev loss

5.2 BiLSTM 训练设置

- 使用 FP16 自动混合精度;
- AdamW 权重衰减为 `1e-4`;
- 根据 dev loss 使用 `ReduceLROnPlateau` 调度器;
- dev loss 连续 2 个 epoch 没有改善时, 学习率乘以 0.5;
- 最低学习率为 `1e-5`;
- 提前停止耐心值为 7 个 epoch。

BiLSTM 在第 30 个 epoch 获得最低 dev WER 并保存为最佳权重。训练日志中的最佳结果为:

```

1 train loss = 0.2074
2 dev loss   = 0.5646
3 dev WER    = 34.05%
4 dev CER    = 12.63%

```

最佳权重保存于:

```

1 /home/zhanz720/library_speech/stage2_outputs_100h_h100/best_model.pt

```

5.3 Transformer 训练设置

- 使用 BF16 自动混合精度;
- AdamW 权重衰减为 `1e-2`;
- 梯度裁剪阈值为 1.0;
- 前 5% optimizer steps 进行线性 warmup;

- warmup 后采用余弦学习率衰减；
- 提前停止耐心值为 10 个 epoch。

Transformer 在第 29 个 epoch 获得最低 dev WER 并保存为最佳权重。训练日志中的最佳结果为：

```
1 | train loss = 0.4721
2 | dev loss   = 0.6890
3 | dev WER    = 45.69%
4 | dev CER    = 15.40%
```

最佳权重保存于：

```
1 | /home/zhanz720/library_speech/transformer_outputs_100h_h100_v2/best_model.pt
```

6. 最终统一评测

训练完成后，使用独立评测脚本加载两个最佳权重。为了消除训练时 FP16 与 BF16 推理精度不同带来的影响，最终评测对两个模型统一采用 FP32，并保证二者使用完全相同的数据顺序、特征、解码器和评价指标实现。

统一评测结果如下。

模型	参数量	dev WER	dev CER	test WER	test CER
BiLSTM-CTC	4,092,701	34.04%	12.63%	33.03%	12.18%
Transformer-CTC	4,123,165	45.76%	15.41%	45.16%	15.24%

训练日志中的 dev 指标与最终统一评测中的 dev 指标存在不超过 0.1 个百分点的微小差异，主要来自训练时混合精度与最终 FP32 推理的数值差异。统一评测的 dev 结果与原训练结果整体一致，说明模型权重、数据读取和评价程序均得到正确复现。

7. 结果分析

7.1 BiLSTM 在当前实验条件下效果更好

在 `test-clean` 上，BiLSTM-CTC 的 WER 为 33.03%，Transformer-CTC 的 WER 为 45.16%。BiLSTM 的 WER 低 **12.13 个百分点**，相对于 Transformer 降低约 **26.9%**。

BiLSTM-CTC 的 CER 为 12.18%，Transformer-CTC 的 CER 为 15.24%。BiLSTM 的 CER 低 **3.06 个百分点**，相对降低约 **20.1%**。

由于两种模型的参数量基本相同，该差异不能简单归因于模型容量。在本实验所采用的 100 小时数据规模、字符级 CTC、从零训练和 greedy 解码条件下，BiLSTM 的序列建模效果优于当前 Transformer 配置。

7.2 验证集与测试集结果一致

BiLSTM 的 test WER 比 dev WER 低 1.01 个百分点，Transformer 的 test WER 比 dev WER 低 0.60 个百分点。两个模型在 `dev-clean` 与 `test-clean` 上的表现接近，且测试集结果没有明显恶化，说明实验结果具有较好的一致性，没有表现出明显的验证集过拟合。

7.3 WER 明显高于 CER

两个模型的 WER 都显著高于 CER。例如，BiLSTM 的 test CER 为 12.18%，但 test WER 为 33.03%。这是因为一个单词中即使只识别错一个字符，该单词也会被计为一个完整的词错误。预测样例中常见错误包括：

- 发音相近字母之间的替换；
- 单词边界，即空格的插入或删除；
- 词尾字符遗漏；
- 连续语音中相邻单词发生粘连。

这些字符级小错误会被 WER 放大，因此 CER 能够更细致地反映基础声学模型已经学到的字符识别能力。

7.4 Transformer 结果较弱的可能原因

Transformer 单样本过拟合测试能够达到 0% WER 和 0% CER，说明模型前向传播、位置编码、padding mask、CTC 目标和解码逻辑能够正常工作。因此，100 小时实验中较高的 WER 更可能是训练和泛化能力问题，而不是明显的程序错误。

可能原因包括：

1. Transformer 通常比循环网络更依赖训练数据量和训练策略；
2. 当前模型从随机初始化开始训练，没有使用自监督预训练表示；
3. 声学前端只有两层简单卷积，没有采用更强的卷积或 Conformer 模块；
4. 输出单位为字符，且只使用 greedy CTC 解码，没有语言模型修正单词拼写和边界；
5. 30 个 epoch 的余弦调度可能还不是该模型的最佳训练方案。

因此，本实验结论应限定为“在当前受控配置下 BiLSTM 优于 Transformer”，而不能推广为“Transformer 不适合语音识别”。

8. 实验结论

本实验完成了基于 LibriSpeech `train-clean-100` 的两种端到端语音识别模型，并在相同条件下进行了完整对比。

最终结果表明，在约 410 万参数、100 小时训练数据、字符级 CTC、无外部语言模型和 greedy 解码的条件下：

- BiLSTM-CTC 在 `test-clean` 上取得 **33.03% WER / 12.18% CER**；
- Transformer-CTC 在 `test-clean` 上取得 **45.16% WER / 15.24% CER**；
- BiLSTM-CTC 在当前实验配置下具有更好的识别性能。

这个结果构成了一组可复现的基础基线。后续如果继续研究 Transformer，可以在保持 `test-clean` 不参与调参的前提下，仅使用 `dev-clean` 比较学习率、训练轮数、模型深度、SpecAugment、子词输出、Conformer 结构或语言模型解码等改进方案。

9. 结果文件位置

统一评测结果保存在服务器目录：

```
1 | /home/zhanz720/library_speech/final_evaluation_100h
```

主要文件包括：

```
1 summary.csv
2 summary.json
3 lstm_dev-clean_predictions.tsv
4 lstm_test-clean_predictions.tsv
5 transformer_dev-clean_predictions.tsv
6 transformer_test-clean_predictions.tsv
```

其中, `summary.csv` 和 `summary.json` 保存汇总指标, 四个 prediction 文件保存每条语音的参考文本和模型预测文本, 可用于进一步分析替换、删除、插入及单词边界错误。